האוניברסיטה הפתוחה

מבוא ל-eBPF

קורס מערכות הפעלה

המחלקה למתמטיקה ולמדעי המחשב

האוניברסיטה הפתוחה

2024

מהדורה פנימית
לא להפצה ולא למכירה
מק"ט 20594-0000

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0005996 belongs to Yair Kurtzman- do not distribute

תנאי שימוש בקובץ הדיגיטלי:

1. הקובץ הוא לשימושך **האישי** בלבד. פרטים מזהים שלך מוטבעים בקובץ בצורה גלויה ובצורה סמויה.

2. השימוש בקובץ הוא אך ורק למטרות לימוד, עיון ומחקר אישי.

3. העתקה או שימוש בתכנים נבחרים מותרת בהיקף העומד בכללי השימוש ההוגן, המפורטים בסעיף 19 לחוק זכות יוצרים 2007. במקרה של שימוש כאמור חלה חובה לציין את מקור הפרסום.

4. הנך רשאי/ת להדפיס דפים מחומר הלימוד לצורכי לימוד, מחקר ועיון אישיים. אין להפיץ או למכור תדפיסים כלשהם מתוך חומר הלימוד.

צוות הפיתוח

כתיבה ועריכה: זאב קליוזנר – חוקר Al & infoSec ומנחה באו״פ
אריה לויטן – מרכז הוראה באו״פ

עריכת לשון: אסנת אפרת

עימוד: אסתר טל

© תשפ״ד – 2024. כל הזכויות שמורות לאוניברסיטה הפתוחה.
ההוצאה לאור של האוניברסיטה הפתוחה, הקריה ע״ש דורותי דה רוטשילד, דרך האוניברסיטה 1, ת״ד 808, רעננה 4353701.
Academic Development and Publishing, The Open University of Israel, The Dorothy de Rothschild Campus, 1 University Road, P.O.Box 808, Raanana 4353701.
Printed in Israel.
אין לשכפל, להעתיק, לצלם, להקליט, לתרגם, לאחסן במאגר מידע, לשדר או לקלוט בכל דרך או בכל אמצעי אלקטרוני, אופטי, מכני או אחר כל חלק שהוא מהחומר שבספר זה. שימוש מסחרי בחומר הכלול בספר זה אסור בהחלט, אלא ברשות מפורשת ובכתב ממדור זכויות יוצרים של האוניברסיטה הפתוחה.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair Kurtzman

File #0005996 belongs to Yair Kurtzman- do not distribute

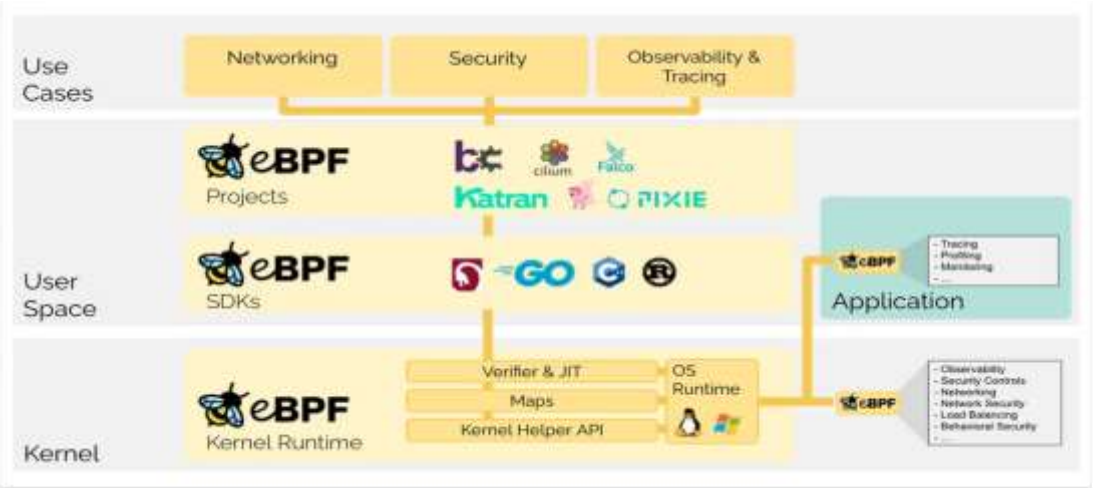
תוכן העניינים

1. מבוא 5
2. ארכיטקטורת eBPF..... 7
3. כתיבת תוכניות eBPF 9
4. אתגרים ומגבלות..... 12
5. סיכום 14
6. נספחים 15
7. רשימת מקורות..... 23

1. מבוא

1.1 מה זה eBPF?

eBPF, ראשי תיבות של extended Berkeley Packet Filter, היא טכנולוגיה מתקדמת המאפשרת הרצת קוד מותאם אישית בתוך ליבת מערכת ההפעלה בצורה יעילה ובטוחה. בתחילה הומצא BPF ככלי לסינון חבילות רשת (packets) ב-Unix, אך עם הזמן התפתח למערכת גמישה ומתקדמת המאפשרת ניטור, אבטחה וניהול משאבים בצורה דינמית ושקופה.
eBPF מאפשר למתכנתים לכתוב תוכניות קטנות אשר רצות בתוך ליבת מערכת ההפעלה ללא צורך בשינוי הקוד המקורי של הליבה עצמה. תכונות אלה הפכו את eBPF לכלי חשוב בניטור ביצועים, אבטחת מידע וניתוח רשתות.



מקור: ebpf.io

1.2 חשיבות eBPF

החשיבות של eBPF נובעת ממספר יתרונות מרכזיים שהוא מציע:

1. **ביצועים גבוהים**: eBPF מאפשר הרצת קוד בתוך הליבה עצמה, יכולת שמובילה לביצועים גבוהים וניטור בזמן אמת, זאת לעומת הגישות המסורתיות שדורשות יציאה לחלל המשתמש.
2. **ביטחון ואבטחה**: תוכניות eBPF עוברות וידוא ובדיקות קפדניות באמצעות מנגנון ה-verifier של ליבת הלינוקס, המבטיח שאין תקלות או פרצות אבטחה.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair Kurtzman

File #0005996 belongs to Yair Kurtzman- do not distribute

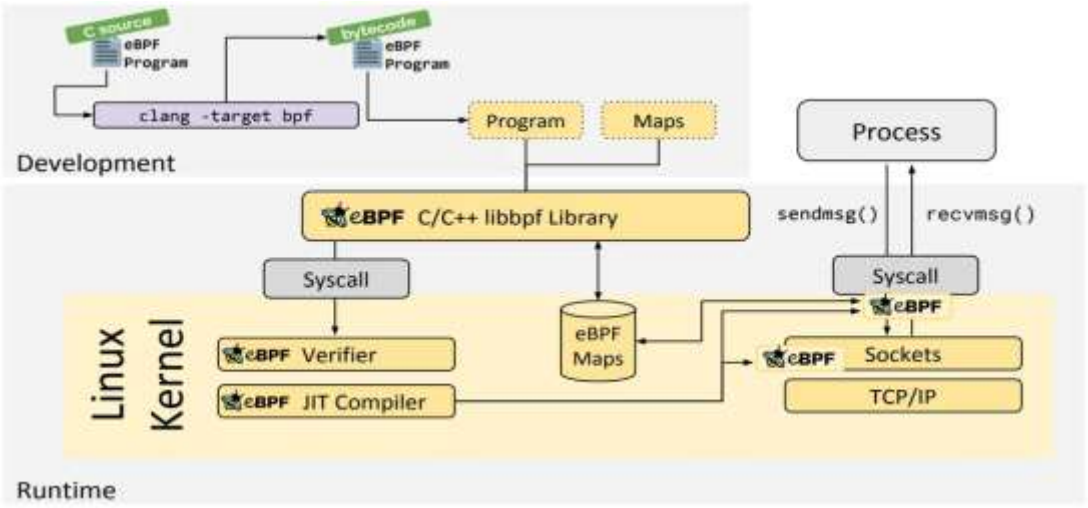
3. **גמישות**: ניתן לשנות ולהרחיב את התוכנות הרצות בליבה מבלי להפסיק את הפעולה של המערכת או לבצע אתחול, יכולת שמאפשרת התאמה מהירה לשינויים ולצרכים חדשים.
4. **שימושים מגוונים**: eBPF משמש במגוון רחב של תחומים, כולל ניטור מערכת (system tracing), אבטחת מידע (security monitoring), ניתוח רשתות (network analysis) ועוד.

eBPF מאפשר ליצור פתרונות המותאמים אישית לצרכים ספציפיים תוך שמירה על ביצועים גבוהים ואבטחה מירבית. בזכות יכולות אלה הפך eBPF לאחד הכלים החזקים והמבוקשים ביותר בתחום מערכות ההפעלה וניהול התשתיות המודרניות.

2. ארכיטקטורת eBPF

2.1 סקירה כללית של הארכיטקטורה

הארכיטקטורה של eBPF בנויה כך שהיא מאפשרת להפעיל קוד מותאם אישית בתוך ליבת מערכת ההפעלה באופן יעיל ובטוח. מערכת eBPF מורכבת ממספר רכיבים עיקריים, כולל מכונה וירטואלית (VM), מסנן מאמת (verifier), ומערכת של hooks המאפשרת לחבר את תוכניות ה-eBPF לנקודות שונות בליבה. רכיבים אלה עובדים יחד כדי לאפשר הרצת קוד מותאם אישית מבלי לסכן את יציבות המערכת או את אבטחתה.



מקור: ebpf.io

2.2 eBPF VM (Virtual Machine)

המכונה הווירטואלית של eBPF היא ליבת הטכנולוגיה. מדובר במכונה וירטואלית פשוטה אך יעילה, המאפשרת להריץ קוד בינארי בתוך הליבה. הקוד נכתב בשפת eBPF, שהיא גרסה מצומצמת של שפת מכונה, המתורגמת בדרך כלל משפת C בעזרת כלים כמו clang/LLVM. ה-eBPF VM נבנה כך שיהיה קל ופשוט ויכיל מספר מוגבל של פקודות. הוא פועל בצורה סינכרונית ומאפשר ביצועים גבוהים תוך שמירה על בטיחות:

- סט פקודות קטן וממוקד: כולל פעולות בסיסיות, כגון חיבור, חיסור והשוואות.
- אכיפת מגבלות גישה וזמן ריצה: כולל שימוש במשתנים מוגדרים מראש וניהול זיכרון מבוקר.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair Kurtzman

File #0005996 belongs to Yair Kurtzman- do not distribute

- תמיכה בהרצת לולאות מוגבלות: הימנעות מלולאות אינסופיות כדי למנוע תקלות.

2.3 תשתית Kernel Hooks

אחד ההיבטים החשובים של eBPF הוא יכולתו להתחבר לנקודות שונות בליבת מערכת ההפעלה באמצעות מערכת hooks. מערכת זו מאפשרת לחבר תוכניות eBPF לנקודות מפתח במערכת, כמו רשת, קבצים ותהליכים, ולבצע ניטור ופעולות מותאמות אישית:

- Tracepoints: נקודות קבועות מראש בליבה שיכולות להפעיל תוכניות eBPF כדי לאסוף נתונים או לבצע פעולות.
- Kprobes ו-Uprobes: מאפשרים לתפוס קריאות פונקציות בתוך הליבה (kprobes) או בתוכניות משתמש (uprobes).
- Socket filters: מאפשרים סינון והפעלה של תוכניות eBPF על תעבורת רשת נכנסת ויוצאת.
- cgroup/bpf: מאפשרים לבצע ניטור וניהול משאבים בתהליכים ובקבוצות תהליכים.

2.4 מאמת eBPF (Verifier)

המאמת (verifier) הוא רכיב קריטי בארכיטקטורת eBPF, שתפקידו לוודא את תקינות ובטיחות הקוד לפני הרצתו בליבה. הוא מבצע בדיקות מקיפות כדי להבטיח שהקוד לא יגרום לתקלות או ייתן גישה לא מבוקרת למשאבי המערכת:

- **בדיקת בטיחות זיכרון**: מוודא שאין גישה לזיכרון לא מאושר.
- **אכיפת כללים טכניים**: מוודא שהקוד לא מכיל לולאות אינסופיות או גישה בלתי חוקית למשתנים.
- **בדיקת קוד מבוקר**: מוודא שהקוד נכתב בהתאם לכללים ולהגבלות של שפת eBPF.

2.5 מנגנון טעינה והרצה

מנגנון הטעינה של eBPF כולל את התהליכים הבאים:

- **טעינת תוכניות eBPF**: תוכניות eBPF נטענות לזיכרון הליבה באמצעות קריאות מערכת (syscalls) מתאימות.
- **הרצה בזמן אמת**: תוכניות אלו רצות בנקודות hook המוגדרות מראש, אוספות נתונים או מבצעות פעולות בזמן אמת.

באופן כללי, ארכיטקטורת eBPF מאפשרת הרצה של קוד מותאם אישית בליבה בצורה בטוחה ויעילה, וכך מתאפשר מגוון רחב של יישומים מתקדמים, כמו ניטור ביצועים, אבטחת מידע וניתוח רשתות.

3. כתיבת תוכניות eBPF

3.1 כלים וספריות

כדי לכתוב ולהריץ תוכניות eBPF יש צורך במספר כלים וספריות המאפשרים תרגום, טעינה וניהול של התוכניות הללו בתוך ליבת מערכת ההפעלה:

- clang/LLVM: הכלים העיקריים לקומפילציה של קוד eBPF. ניתן לכתוב את התוכניות בשפת C ולתרגם אותן באמצעות clang ל-bytecode המותאם ל-eBPF VM.
- bpftrace: כלי רב-עוצמה שמאפשר כתיבת סקריפטים קצרים ויעילים ב-eBPF תוך שימוש בתחביר דמוי DTrace. כלי זה משמש במיוחד לניטור ואבחון מערכות.
- BCC (BPF Compiler Collection): אוסף של כלים וספריות לכתיבה, קומפילציה, טעינה וניהול של תוכניות eBPF. BCC מספקת גם ממשקי API ב-Python ובשפות נוספות, שמקלים על פיתוח תוכניות מורכבות.

3.2 מבנה התוכנית

תוכנית eBPF טיפוסית כוללת מספר חלקים עיקריים: כתיבת הקוד, קומפילציה, טעינה והרצה. להלן דוגמה בסיסית למבנה תוכנית eBPF:

כתיבת הקוד

תוכנית eBPF נכתבת בשפת C וכוללת פונקציות המוגדרות כקוד eBPF.

```
#include <uapi/linux/ptrace.h>
#include <linux/sched.h>
BPF_HASH(start, u32);
int do_trace(struct pt_regs *ctx, struct task_struct *task) {
    u32 pid = bpf_get_current_pid_tgid();
    start.update(&pid, &task->pid);
    return 0;
}
```

קומפילציה

משתמשים ב-clang כדי לתרגם את הקוד ל-bytecode של eBPF.

```
clang -O2 -target bpf -c trace.c -o trace.o
```

טעינה והרצה

יש לטעון את התוכנית לליבת מערכת ההפעלה ולהגדיר את הנקודה שבה היא תפעל (hook).

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0005996 belongs to Yair Kurtzman- do not distribute


```
from bcc import BPF
b = BPF(src_file="trace.c")
b.attach_kprobe(event="do_sys_open", fn_name="do_trace")
```

3.3 ניפוי באגים ואופטימיזציה

ניפוי באגים ואופטימיזציה של תוכניות eBPF הם שלבים חשובים בתהליך הפיתוח, כדי להבטיח שהקוד יעיל ואינו מכיל שגיאות שעלולות לגרום לתקלות במערכת.

כלים לניפוי באגים:

- **bpftool**: כלי רב-עוצמה לניהול וניפוי באגים של תוכניות eBPF. כלי זה מאפשר לראות את התוכניות הטעונות, להציג את הקוד המורכב ולעקוב אחר הביצועים.
- **bpftrace**: מאפשר כתיבת סקריפטים לבדיקות וניפוי באגים בצורה מהירה ויעילה.
- **perf**: כלי נוסף לניטור וניפוי ביצועים של תוכניות eBPF.

טיפים לאופטימיזציה:

- **צמצום מספר הפעולות**: יש להימנע מקוד מורכב וארוך בתוך הפונקציות של eBPF, כדי לשמור על ביצועים גבוהים.
- **שימוש יעיל בזיכרון**: חשוב לוודא שאין דליפות זיכרון ולשמור על שימוש מינימלי במשאבים.
- **מעקב אחרי ביצועים**: שימוש בכלים כמו bpftool ו-perf לבדיקת ביצועים ואיתור צווארי בקבוק.

דוגמה מורכבת יותר

ניקח לדוגמה תוכנית eBPF שמנטרת קריאות לכתיבת קבצים ומדווחת על קריאות לכתיבה בקבצים מסוימים:

כתיבת הקוד

```
 include <uapi/linux/ptrace.h>
 include <linux/fs.h>
BPF_PERF_OUTPUT(events);
struct data_t {
   u32 pid;
   char comm[TASK_COMM_LEN];
   char filename[NAME_MAX];
};
int trace_write(struct pt_regs *ctx, struct file *file) {
   struct data_t data = {};
```

```
    u32 pid = bpf_get_current_pid_tgid();
    data.pid = pid;
    bpf_get_current_comm(&data.comm, sizeof(data.comm));
    bpf_probe_read_kernel(&data.filename, sizeof(data.filename),
file->f_path.dentry->d_name.name);
    events.perf_submit(ctx, &data, sizeof(data));
    return 0;
}
```

קומפילציה

```
clang -O2 -target bpf -c write_trace.c -o write_trace.o
```

טעינה והרצה

```
from bcc import BPF
def print_event(cpu, data, size):
    event = b["events"].event(data)
    print("%-18s %-16s %s" % (event.pid, event.comm, event.filename))
b = BPF(src_file="write_trace.c")
b.attach_kprobe(event="vfs_write", fn_name="trace_write")
b["events"].open_perf_buffer(print_event)
while True:
    b.perf_buffer_poll()
```

תוכנית זו תתפוס קריאות לכתיבת קבצים ותדווח על שמות הקבצים, PID ותהליך המשתמש.

באמצעות הכלים והטכניקות המוזכרים כאן ניתן לכתוב, לנפות ולבצע אופטימיזציה לתוכניות eBPF בצורה יעילה, וכך לאפשר ניצול היכולות המתקדמות של טכנולוגיה זו לצורכי ניטור, אבטחה וניהול מערכות.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0005996 belongs to Yair Kurtzman- do not distribute

4. אתגרים ומגבלות

4.1 מגבלות טכניות

למרות היתרונות הרבים של eBPF, יש לו גם מספר מגבלות טכניות שיש לקחת בחשבון בעת השימוש בטכנולוגיה זו:

- **מורכבות הפיתוח**: כתיבת תוכניות eBPF דורשת הבנה מעמיקה של ליבת מערכת ההפעלה וקריאות מערכת. עבור מפתחים שאינם מנוסים בתכנות ליבה או בכתיבת קוד ברמת מערכת ההפעלה, תהליך הפיתוח יכול להיות מאתגר.
- **מגבלות זיכרון**: תוכניות eBPF פועלות בתוך ליבת מערכת ההפעלה, ויש להן מגבלות על כמות הזיכרון שהן יכולות להשתמש בו. על מנת לשמור על יציבות המערכת, יש לוודא שהתוכניות לא צורכות יותר מדי משאבים.
- **אבטחה ובדיקות**: למרות ש-eBPF יכול לשמש לאבטחת המערכת, תוכניות eBPF עצמן צריכות להיבדק בקפידה כדי להבטיח שאין בהן חולשות אבטחה שעלולות לחשוף את המערכת להתקפות.
- **תמיכה במערכת ההפעלה**: לא כל מערכות ההפעלה תומכות ב-eBPF באופן מלא, ויש לוודא שהגרסה של מערכת ההפעלה נמצאת בגרסה התומכת ב-eBPF.

4.2 אתגרי ביצועים

שימוש ב-eBPF יכול להוות אתגר מבחינת ביצועים:

- **עומס נוסף על המערכת**: בעוד ש-eBPF ידוע בכך שהוא קל משקל יחסית, עדיין ישנו עומס נוסף שמתווסף למערכת. יש להבטיח שהתוכניות לא מכבידות על המערכת בצורה משמעותית, במיוחד במערכות בעלות עומס גבוה.
- **השפעה על זמני תגובה**: תוכניות eBPF שמבצעות פעולות כבדות או לא אופטימליות עלולות להוסיף עיכובים לזמני התגובה של המערכת. יש לכתוב תוכניות יעילות ולבצע אופטימיזציות במידת הצורך.

4.3 אתגרי אבטחה

למרות היתרונות בתחום האבטחה, גם כאן יש אתגרים:

- **אכיפת מדיניות אבטחה**: יש לוודא שתוכניות eBPF שנכתבות לא עוקפות מדיניות אבטחה קיימות ולא יוצרות פרצות חדשות. כלים ונהלים לאבטחה צריכים להיות מוטמעים בתהליך הפיתוח והפריסה.
- **גישה לרכיבי ליבה**: מכיוון שתוכניות eBPF פועלות ברמת הליבה, כל טעות בקוד עלולה לגרום לבעיות קשות במערכת, כולל קריסות. חשוב לבדוק היטב את הקוד ולבצע בדיקות מקיפות.

4.4 ניפוי באגים ואבחון

ניפוי באגים ואבחון תוכניות eBPF הם תהליכים מורכבים:

- **קושי בניפוי באגים**: ניפוי באגים של תוכניות eBPF יכול להיות מאתגר, בעיקר בגלל שהן פועלות בליבת מערכת ההפעלה. יש צורך בכלים מתקדמים ובידע מעמיק לניפוי באגים בצורה יעילה.
- **מגבלות תשתית**: במערכות מורכבות ומבוזרות ישנם מקרים שבהם קשה לאתר את מקור הבעיה. שימוש בכלי ניתוח כמו bpftool ו-perf יכול לעזור, אך עדיין נדרש ידע נרחב בתחום.

4.5 תאימות ועדכונים

התאימות עם גרסאות שונות של מערכת ההפעלה ועדכונים מהווים אתגר נוסף:

- **עדכוני ליבה**: יש לוודא שהתוכניות תואמות לעדכונים בגרסת הליבה של מערכת ההפעלה. עדכונים ושינויים בליבת מערכת ההפעלה יכולים להשפיע על פעולת תוכניות eBPF ולדרוש שינויים בקוד.
- **תמיכה לאחור**: לעתים יש צורך בתמיכה במערכות הפעלה ישנות יותר שאינן תומכות באופן מלא ב-eBPF. במקרים כאלה יש למצוא פתרונות חלופיים או לבצע התאמות מיוחדות.

למרות האתגרים והמגבלות, השימוש ב-eBPF ממשיך לצמוח ולהתפתח, ומספק כלים חזקים לניטור, לאבטחה ולניהול מערכות הפעלה מודרניות. חשוב להיות מודעים למגבלות אלו ולפעול לצמצום הסיכונים והקשיים על ידי שימוש בכלים מתקדמים, בפרקטיקות מיטביות ובלמידה מתמשכת.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0005996 belongs to Yair Kurtzman- do not distribute

5. סיכום

eBPF (Extended Berkeley Packet Filter) הוא טכנולוגיה פורצת דרך המספקת יכולות ניטור, אבטחה וניהול מתקדמות למערכות הפעלה מודרניות. מאז הצגתו הפך eBPF לכלי מרכזי עבור מפתחים ומהנדסים המבקשים לשפר את הביצועים ואת האמינות של המערכות שלהם תוך שמירה על אבטחה גבוהה.

נקודות המפתח:

- ארכיטקטורה: היכולת להריץ קוד מותאם אישית בתוך ליבת מערכת ההפעלה בצורה בטוחה ויעילה הופכת את eBPF לכלי רב-עוצמה למגוון רחב של יישומים.
- יישומים: eBPF משמש בתחומים רבים, כולל ניטור ביצועים, אבטחה, ניהול רשתות ופרופילינג של מערכות. השימוש בו מאפשר גישה מדויקת ויעילה לנתוני מערכת קריטיים.
- כתיבת תוכניות: כתיבת תוכניות eBPF דורשת ידע טכני מתקדם, אך כלים כמו BCC, bpftrace ו-Cilium מקלים על התהליך ומאפשרים פיתוח מהיר ויעיל.
- אינטגרציה עם **Kubernetes** ו-**eBPF:** Docker מספק פתרונות מתקדמים לניטור ואבטחה בסביבות קונטיינרים, ותורם לשיפור הביצועים והאבטחה בסביבות מורכבות ומבוזרות.
- אתגרים ומגבלות: למרות היתרונות, ישנם אתגרים טכניים, אבטחתיים ותפעוליים שיש להתגבר עליהם. התאמות ושיפורים מתמשכים נדרשים כדי להבטיח את יעילות הטכנולוגיה.
- עתיד הטכנולוגיה: עם התפתחות הכלים והתמיכה בקהילת המפתחים, eBPF צפוי להמשיך להתפתח, להתרחב לתחומים חדשים ולשפר את היכולות של מערכות הפעלה מודרניות.

eBPF משנה את האופן שבו אנו מנטרים ומאבטחים מערכות הפעלה ויישומים, ומספק כלים רבי-עוצמה למפתחים ולמהנדסים. למרות האתגרים והמגבלות, היתרונות המשמעותיים של הטכנולוגיה הזאת הופכים אותה לכלי בלתי נפרד בארסנל של אנשי טכנולוגיה רבים. העתיד של eBPF מבטיח חדשנות והרחבת היכולות, שיאפשרו לנו להתמודד בצורה טובה יותר עם האתגרים הטכנולוגיים המודרניים.

6. נספחים

6.1 יישומים של eBPF

6.1.1 ניטור מערכת

אחד השימושים המרכזיים והחשובים של eBPF הוא בניטור מערכות. יכולת זו מאפשרת למנהלי מערכות ולמתכנתים לאסוף נתונים בזמן אמת על התנהגות המערכת, לזהות בעיות ולבצע אופטימיזציות.



מקור: ebpf.io

- ניטור ביצועים: eBPF מאפשר ניטור מדויק של ביצועי המערכת, כולל מעקב אחרי צריכת CPU, זיכרון ו-I/O של תהליכים שונים. דוגמאות לכלים המשתמשים ב-eBPF לניטור ביצועים כוללים את bpftrace, perf ו-sysdig.
- אבחון בעיות: באמצעות eBPF ניתן לזהות צווארי בקבוק ובעיות אחרות במערכת, כמו תהליכים שגורמים לעיכובים או לקריאות מערכת שגויות. ניתן לכתוב סקריפטים מותאמים אישית לאבחון בעיות אלו ולהפעילם בזמן אמת כדי לקבל תובנות מיידיות.
- מעקב אחר תעבורת מערכת: eBPF יכול לשמש למעקב אחר תעבורת מערכת, כולל קריאות קבצים, שימוש ברשת ופעולות תהליכים, יכולת שמספקת תמונה מקיפה של פעילות המערכת.

6.1.2 אבטחת מידע

eBPF מאפשר גם חיזוק אבטחת המידע במערכת על ידי זיהוי איומים וניתוח תעבורה בזמן אמת.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0005996 belongs to Yair Kurtzman- do not distribute

- זיהוי איומים: ניתן לכתוב תוכניות eBPF שמנטרות פעילות חשודה במערכת, כמו ניסיונות גישה לא מורשים או קריאות מערכת חריגות. תוכניות אלו יכולות לפעול כפרואקטיביות ולהתריע או לחסום את הפעולה.
- ניתוח תעבורה: eBPF מאפשר ניתוח מדויק של תעבורת רשת, תכונה שיכולה לעזור בזיהוי תקיפות רשת (כגון DDoS) ופעילויות חשודות נוספות. ניתוח זה נעשה באמצעות חיבור לסוקטים ולנקודות hook ברשת.
- ניהול הרשאות ואכיפה: ניתן להשתמש ב-eBPF כדי לנהל הרשאות גישה למשאבים שונים במערכת ולאכוף מדיניות אבטחה ברמת הליבה, וכך להבטיח שהמערכת תישאר בטוחה.

6.1.3 רשתות

שימוש נוסף וחשוב של eBPF הוא בתחום הרשתות, שם הוא מאפשר יישום ניתוח ותפעול מתקדם של תעבורת רשת.

- ניתוב מתקדם: eBPF מאפשר יצירת מסלולים מותאמים אישית לתעבורת רשת, ניתוח וביצוע ניתוב מתקדם בהתאם לצרכים ספציפיים – כולל ניתוב דינמי, ניתוב מבוסס מדיניות ועוד.
- ניתוח פרוטוקולים: ניתן לכתוב תוכניות eBPF לניתוח ולבדיקת פרוטוקולים שונים, וכך לסייע בזיהוי תקלות, באבחון בעיות ובשיפור ביצועי רשת.
- שיפור ביצועים: באמצעות ניטור וניהול תעבורת הרשת בצורה חכמה, ניתן לשפר את ביצועי הרשת הכוללים, להקטין את העומס על שרתים ולהבטיח חוויית משתמש טובה יותר.

6.1.4 אובזרבביליות (Observability)

eBPF תורם רבות גם לאובזרבביליות, המאפשרת קבלת תמונה מלאה וברורה על פעילות המערכת והיישומים הרצים עליה.

- **איסוף נתונים בזמן אמת**: ניתן לאסוף נתונים בזמן אמת על ביצועי היישומים, תהליכים ושירותי מערכת, ולבצע ניתוחים מעמיקים כדי להבין את התנהגות המערכת.
- **תמיכה במערכות מבוזרות**: eBPF מאפשר ניטור ואיסוף נתונים גם במערכות מבוזרות כמו Kubernetes ו-Docker, יכולת המספקת כלי עזר רב עוצמה בניהול ובתפעול של סביבות קונטיינרים.

6.1.5 אינטגרציה עם כלי DevOps

eBPF משתלב היטב עם כלי DevOps, ומאפשר לאנשי DevOps ולמפתחים לכתוב תוכניות ניטור ואבטחה מותאמות אישית המשתלבות בתהליכי CI/CD.

- **ניטור שירותים**: באמצעות eBPF ניתן לכתוב תוכניות לניטור שירותים בענן ובשרתים מקומיים, ולשלב אותן במערכות ניהול תצורה וכלי אוטומציה כמו Ansible, Puppet ו-Chef.

- **אוטומציה של אבטחה**: ניתן להשתמש ב-eBPF לאוטומציה של תהליכי אבטחה, כמו ניתוח לוגים וזיהוי פעילות חשודה, ובכך לשפר את היכולת להגיב במהירות לאיומים.

באמצעות eBPF ניתן ליישם ולנהל פתרונות מורכבים ויעילים לניטור, לאבטחה, ולניהול רשתות במערכות הפעלה מודרניות, וכך הביצועים, האבטחה ואמינות המערכת משתפרים.

6.2 אינטגרציה עם Kubernetes ו-Containerization

6.2.1 eBPF ו-Kubernetes

Kubernetes הוא כלי נפוץ לניהול קונטיינרים, המאפשר אורקסטרציה אוטומטית של יישומים בכלי קונטיינר כמו Docker. eBPF משתלב בצורה מושלמת בסביבות Kubernetes כדי לשפר את יכולות הניטור, האבטחה והביצועים של יישומים וקונטיינרים.

- **ניטור קונטיינרים**: בעזרת eBPF ניתן לנטר את כל הקונטיינרים הרצים בסביבת Kubernetes בצורה מדויקת ויעילה. ניתן לאסוף נתונים על ביצועים, שימוש במשאבים וקריאות מערכת מבלי לפגוע בביצועים של היישומים הרצים.
- **אבטחה**: ניתן להשתמש ב-eBPF לניתור תעבורת רשת בין קונטיינרים ולזיהוי פעילויות חשודות בזמן אמת. שימושים אלה מאפשרים גילוי של התקפות כמו ניסיונות חדירה ונוזקות והגנה מפניהן.
- **תשתיות רשת מתקדמות**: eBPF מאפשר להגדיר מדיניות ניתוב מתקדמת ולבצע אופטימיזציה של תעבורת הרשת בסביבת Kubernetes, כולל שימוש בכלים כמו Cilium, אשר משתמש ב-eBPF לניהול רשתות ב-Kubernetes.

דוגמה לשימוש ב-Cilium

Cilium הוא פרויקט קוד פתוח המשתמש ב-eBPF לניהול רשתות ואבטחה בסביבת Kubernetes.



מקור: github.com/cilium/cilium

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair Kurtzman

File #0005996 belongs to Yair Kurtzman- do not distribute

התקנת Cilium:

kubectl create -f
https://raw.githubusercontent.com/cilium/cilium/v1.9/install/kubernetes/quick-install.yaml

ניטור תעבורה עם Hubble:

Hubble הוא כלי מבוסס Cilium המספק נראות (observability) לרשת Kubernetes באמצעות eBPF.

hubble enable
hubble observe

6.2.2 eBPF ו-Docker

Docker הוא כלי פופולרי נוסף לניהול קונטיינרים. eBPF משתלב היטב עם Docker ומאפשר ניהול מתקדם של משאבים וניטור בתוך קונטיינרים.

- **ניטור ומשאבים**: באמצעות eBPF ניתן לנטר את הביצועים והשימוש במשאבים של קונטיינרים בודדים בתוך Docker. ניתן לאסוף נתונים על צריכת CPU, זיכרון ו-I/O, ולזהות צווארי בקבוק או בעיות אחרות.
- **אבטחת קונטיינרים**: eBPF מאפשר לזהות ולמנוע התקפות בתוך קונטיינרים על ידי ניתוח תעבורת רשת ופעילויות מערכת חשודות.
- **שיפור ביצועים**: eBPF מאפשר לבצע אופטימיזציה לניהול המשאבים והביצועים של הקונטיינרים, כולל קביעת מדיניות ניתוב מתקדמת ותמיכה בפרוטוקולים מורכבים.

דוגמה לניטור קונטיינרים עם BCC

BCC (BPF Compiler Collection) הוא אוסף כלים המאפשר לכתוב תוכניות eBPF ולנהל אותן. בעזרת BCC ניתן לנטר קונטיינרים בסביבת Docker.

התקנת BCC:

sudo apt-get install bpfcc-tools linux-headers-$(uname -r)

ניטור ביצועים בתוך קונטיינרים:

sudo /usr/share/bcc/tools/execsnoop -t -x -c

ניטור רשת בתוך קונטיינרים:

sudo /usr/share/bcc/tools/tcpretrans -c

6.2.3 יתרונות האינטגרציה

השילוב של eBPF עם Kubernetes ו-Docker מביא עמו מספר יתרונות מובהקים:

- **ביצועים גבוהים**: ניטור ואבטחה בזמן אמת מבלי לפגוע בביצועים של היישומים ושל הקונטיינרים.
- **גמישות ונראות**: נראות טובה יותר על פעילות המערכת ותעבורת הרשת, עם יכולת להתאים את הניטור ואת האבטחה לצרכים הספציפיים של הארגון.
- **תגובה מהירה לאיומים**: יכולת לזהות ולהגיב במהירות לאיומים ולאירועים חשודים בסביבת הקונטיינרים.

באמצעות eBPF ניתן להרחיב את יכולות הניטור, האבטחה והניהול של Kubernetes ו-Docker בצורה משמעותית, וכך ישתפרו הביצועים, האבטחה והאמינות של מערכות מבוזרות ושל קונטיינרים.

6.3 מקרים לדוגמה (Case Studies)

6.3.1 ניטור ביצועים בחברת SaaS

חברת SaaS בינלאומית המתמחה בשירותי ענן נתקלה בבעיות ביצועים בלתי מוסברות בשירותיה. מהנדסי החברה החליטו להשתמש ב-eBPF לניטור ואבחון הבעיות.

- **האתגר**: החברה נתקלה בצווארי בקבוק שגרמו לעיכובים בטעינת דפי אינטרנט ובתגובה לשירותים.
- **הפתרון**: באמצעות כלי eBPF, כמו bpftrace ו-BCC, כתבו מהנדסי החברה סקריפטים שניטרו את קריאות המערכת, שימוש במשאבים ותעבורת רשת.
- **התוצאות**: הצוות זיהה קריאות מערכת איטיות ותהליכים שצרכו משאבים רבים. לאחר תיקון הקוד ואופטימיזציה של התהליכים חוותה החברה שיפור ניכר בביצועים וזמני תגובה מהירים יותר.

6.3.2 אבטחת רשת בחברת פיננסים

חברת פיננסים גדולה התמודדה עם אתגרים בניטור ובאבטחת תעבורת הרשת הפנימית שלה, במיוחד בזיהוי ובאיתור איומים בזמן אמת.

- **האתגר**: החברה נזקקה לכלי אבטחה שיוכל לזהות ולחסום פעילות חשודה מבלי לפגוע בביצועים.
- **הפתרון**: החברה השתמשה ב-Cilium, פרויקט מבוסס eBPF, לניטור ולאבטחת תעבורת הרשת בתוך Kubernetes. תוכניות eBPF שנכתבו במטרה לזהות דפוסים של התקפות וסריקות פורטים הוטענו לתוך הליבה.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0005996 belongs to Yair Kurtzman- do not distribute

- **התוצאות**: החברה הצליחה לזהות ולחסום מספר התקפות בזמן אמת מבלי לפגוע בביצועי הרשת או השירותים הרצים. מערכת האבטחה החדשה שיפרה את היכולת להגן על נתוני הלקוחות ועל המערכות הפנימיות של החברה.

6.3.3 אופטימיזציה של תשתיות רשת בחברת טכנולוגיה

חברת טכנולוגיה בינלאומית פעלה לשיפור תשתיות הרשת שלה על מנת לתמוך בעומסים גוברים ובדרישות ביצועים גבוהות.

- **האתגר**: תשתיות הרשת של החברה נתקלו בעומסים כבדים שגרמו להאטות ולשיבושים בתפקוד השירותים.
- **הפתרון**: באמצעות שימוש ב-eBPF לניטור ולניתוח תעבורת רשת, הצליח הצוות הטכני לנתח את דפוסי השימוש ברשת ולזהות צווארי בקבוק. כלים כמו bpftool ו-perf שימשו לאיסוף נתונים מדויקים על ביצועי הרשת.
- **התוצאות**: על ידי אופטימיזציה של הנתיבים והשירותים ברשת הצליחה החברה לשפר את קיבולת הרשת ולהפחית את זמני ההמתנה. כתוצאה מכך חוו הלקוחות שיפור משמעותי בביצועים ובאמינות השירותים.

6.3.4 שיפור נראות ואובזרבביליות בחברת E-commerce

חברת E-commerce גלובלית התמודדה עם אתגרים בקבלת נראות מלאה על מערכותיה המורכבות והמבוזרות.

- **האתגר**: קושי באיסוף נתונים על ביצועי המערכת וקריאות השירותים בזמן אמת, במיוחד בסביבה מבוזרת עם מאות קונטיינרים.
- **הפתרון**: החברה יישמה פתרונות eBPF, כמו BCC ו-bpftrace, ליצירת סקריפטים מותאמים אישית שאספו נתונים קריטיים בזמן אמת. הסקריפטים נכתבו כך שינטרו תהליכים, קריאות מערכת ותעבורת רשת.
- **התוצאות**: החברה הצליחה לשפר את הנראות של פעילות המערכת שלה, לזהות בעיות בזמן אמת ולפתור אותן במהירות. שיפור זה הוביל להגברת היעילות התפעולית ולשיפור חוויית הלקוח.

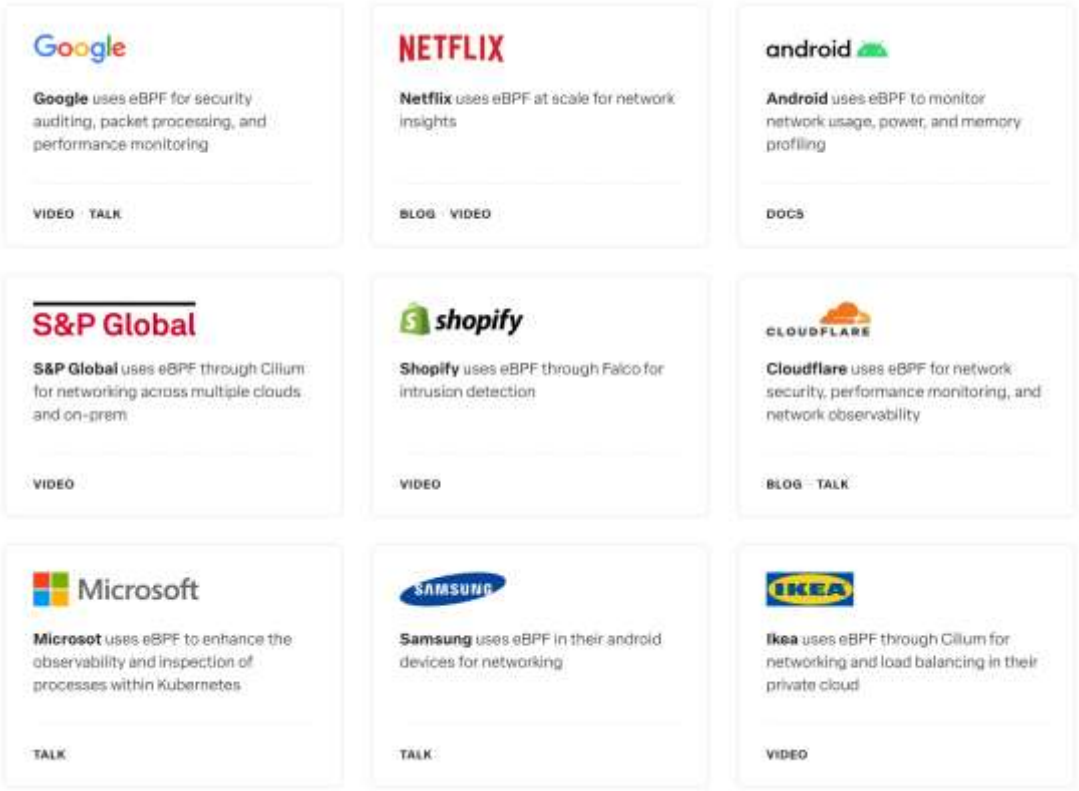
6.3.5 פתרון בעיות ביצועים בקונטיינרים בחברת מדיה

חברת מדיה גדולה נתקלת בבעיות ביצועים בסביבות הקונטיינרים שלה, שמשפיעות על יכולתה לספק תוכן בזמן אמת.

- **האתגר**: בעיות ביצועים בקונטיינרים שהובילו לשיהוי בהזרמת וידאו ותוכן למשתמשים.
- **הפתרון**: באמצעות eBPF וכלי ניטור כמו BCC ו-bpftrace ניטרה החברה את ביצועי הקונטיינרים בזמן אמת. סקריפטים מותאמים זיהו קריאות מערכת איטיות וניצול משאבים חריג.

- **התוצאות**: לאחר אופטימיזציה של תהליכים ושיפור ניהול המשאבים הצליחה החברה להפחית את זמני השיהוי ולשפר את ביצועי השירותים שלה, וכתוצאה מכך השתפרה חוויית המשתמשים במידה ניכרת.

באמצעות היישומים המגוונים של eBPF חברות רבות מצליחות לשפר את ביצועי המערכות שלהן, להגביר את האבטחה ולקבל נראות מלאה של הפעילות שלהן, כל זאת תוך שמירה על ביצועים גבוהים ואמינות מרבית.



מקור: ebpf.io

6.4 עתיד eBPF

6.4.1 התפתחות טכנולוגית

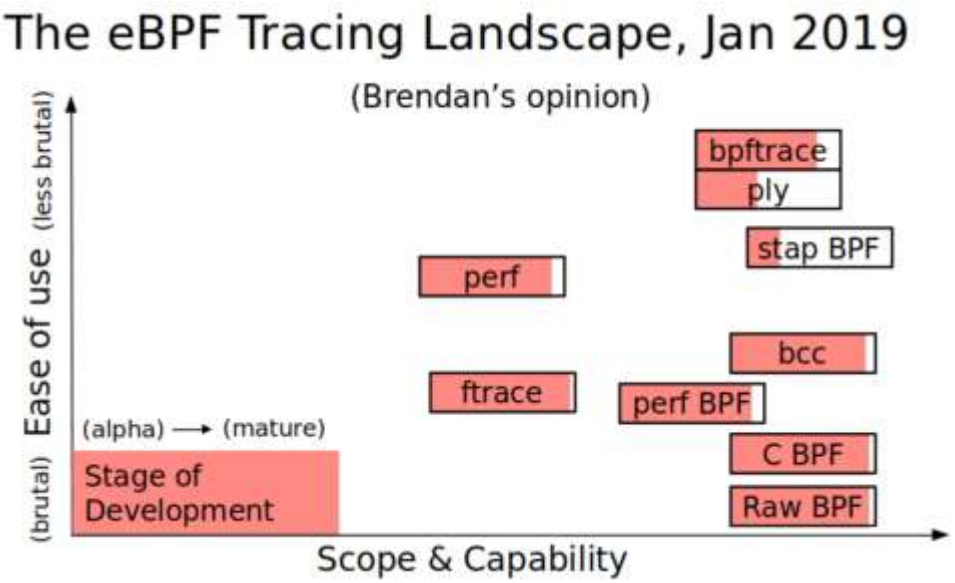
eBPF נמצא כיום בחזית הטכנולוגיה בתחומי הניטור, האבטחה וניהול מערכות הפעלה, וצפוי להמשיך להתפתח בשנים הקרובות. ישנם מספר תחומים שבהם צפויה התקדמות משמעותית:

- **תמיכה רחבה יותר במערכות הפעלה**: כיום eBPF נתמך בעיקר בלינוקס, אך ישנן יוזמות להרחיב את התמיכה גם למערכות הפעלה אחרות כמו Windows ו-macOS. עם הזמן צפויה להיות הרחבה של התמיכה בטכנולוגיה זו בסביבות שונות.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair Kurtzman

File #0005996 belongs to Yair Kurtzman- do not distribute

- **שיפורי ביצועים**: למרות ש-eBPF נחשב ליעיל, תמיד יש מקום לשיפור. עבודה על אופטימיזציות נוספות והפחתת העומס על מערכת ההפעלה תאפשר להרחיב את השימוש ב-eBPF גם בסביבות בעלות עומס כבד במיוחד.
- **כלי פיתוח וניטור מתקדמים**: צפויה יצירה ושיפור של כלים לפיתוח, ניפוי באגים וניטור של תוכניות eBPF, כמו גם יצירת ממשקי משתמש ידידותיים יותר שיאפשרו למפתחים לעבוד בצורה יעילה יותר.



מקור: brendangregg.com/ebpf.html

6.4.2 אינטגרציה עם טכנולוגיות ענן וקונטיינרים

eBPF כבר הוכיח את עצמו ככלי רב עוצמה בסביבות Kubernetes ו-Docker, והעתיד מבשר על אינטגרציה רחבה ומעמיקה יותר:

- **שילוב עם מערכות אורקסטרציה נוספות**: מלבד Kubernetes, צפוי שילוב של eBPF עם מערכות אורקסטרציה נוספות ועם מערכות לניהול ענן, וכך יתאפשרו נראות ואבטחה מתקדמים גם בסביבות מרובות עננים והיברידיות.
- **אבטחה וניטור משולבים**: אינטגרציה עם פתרונות אבטחה וניטור מתקדמים תאפשר ל-eBPF להיות חלק בלתי נפרד ממערכות הגנת סייבר וניהול ביצועים, תוך שיפור היכולת להגיב לאיומים ולהבטיח ביצועים מיטביים בזמן אמת.

6.4.3 קהילת מפתחים וחדשנות

קהילת המפתחים סביב eBPF הולכת וגדלה, וגידול הקהילה מוביל לחדשנות מתמדת:

- **פרויקטים קוד פתוח**: פרויקטים כמו Cilium, bpftrace ו-BCC ממשיכים להתפתח ולספק כלים ושיפורים נוספים. קהילת הקוד הפתוח ממשיכה לתרום ולהרחיב את האפשרויות של eBPF.

- **חינוך והכשרה**: עם התרחבות השימוש ב-eBPF צפויה עלייה בהשקעה בחינוך ובהכשרה של מפתחים בשימוש בטכנולוגיה זו. קורסים, סדנאות וחומרי לימוד נוספים יהיו זמינים למפתחים בכל הרמות.

6.4.4 יישומים חדשים

הפוטנציאל של eBPF רחב מאוד, ועם הזמן צפויים להופיע יישומים חדשים ומרתקים:

- **בינה מלאכותית ולמידת מכונה**: שימוש ב-eBPF לניטור ולאיסוף נתונים מדויק לשם שיפור מודלים של בינה מלאכותית ולמידת מכונה, כמו גם להגנה על מערכות מבוססות AI.
- **אינטרנט של הדברים (IoT)**: ניטור ואבטחה של מכשירי IoT בעזרת eBPF יכולים לשפר את היכולת לנטר מערכות גדולות ומורכבות של מכשירים מחוברים.
- **פיתוח אפליקציות מתקדמות**: שילוב eBPF באפליקציות שמצריכות ניטור בזמן אמת, כמו משחקים מקוונים, אפליקציות פיננסיות ואפליקציות קריטיות אחרות.

6.4.5 סיכום

העתיד של eBPF נראה מבטיח עם פוטנציאל לחדשנות ולשיפור מתמיד במגוון רחב של תחומים. ככל שהטכנולוגיה ממשיכה להתפתח, צפויות להופיע אפשרויות חדשות ויישומים מגוונים שישפיעו על האופן שבו אנו מנטרים, מאבטחים ומנהלים מערכות הפעלה מודרניות וסביבות קונטיינרים. eBPF ממשיך להיות כלי מרכזי במאמץ לשיפור הביצועים והאבטחה של מערכות מורכבות, והציפייה היא לראות אותו בשימוש רחב יותר ובאינטגרציה עמוקה יותר בתשתיות טכנולוגיות רבות.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0005996 belongs to Yair Kurtzman- do not distribute

7. רשימת מקורות

מאמרים ומדריכים

1. eBPF Documentation – https://ebpf.io
   - האתר הרשמי של eBPF, המכיל תיעוד מקיף, מדריכים ומשאבים נוספים.
2. Introduction to eBPF – Brendan Gregg
   - מדריך מקיף שמסביר את יסודות eBPF, שימושים ויישומים מעשיים. זמין באתרו של Brendan Gregg: https://www.brendangregg.com/ebpf.html.
3. BPF and XDP Reference Guide
   - מדריך מקיף לשימוש ב-eBPF וב-XDP, זמין באתר GitHub: https://github.com/xdp-project/xdp-tutorial.

ספרים

1. BPF Performance Tools – Brendan Gregg
   - ספר מקיף על כלים ופרקטיקות לשימוש ב-eBPF לשיפור ביצועים, ניטור ואבטחה: https://www.brendangregg.com/bpf-performance-tools-book.html.
2. Linux Observability with BPF - David Calavera, Lorenzo Fontana
   - ספר שמסביר על כלי ניטור מתקדמים בלינוקס באמצעות eBPF, כולל דוגמאות קוד ושימושים מעשיים.

   Calavera, D., & Fontana, L. (2019). *Linux Observability with BPF: Advanced Programming for Performance Analysis and Networking*. O'Reilly Media.
3. Learning eBPF Programming the Linux Kernel for Enhanced Observability, Networking, and Security
   - https://cilium.isovalent.com/hubfs/Learning-eBPF%20-%20Full%20book.pdf

כלים שימושיים

1. BCC (BPF Compiler Collection)
   - אוסף כלים וממשקים לכתיבת תוכניות eBPF. תיעוד וכלים זמינים ב-GitHub: https://github.com/iovisor/bcc
2. bpftrace
   - כלי שפותח במקור על ידי Brendan Gregg לניטור דינמי של מערכת לינוקס באמצעות eBPF. תיעוד וכלים זמינים ב-GitHub: https://github.com/iovisor/bpftrace.
3. Cilium
   - פתרון רשתות ואבטחה מבוסס eBPF לסביבות Kubernetes. תיעוד וכלים זמינים באתר הרשמי: https://cilium.io.
4. Hubble
   - כלי נראות רשת מבוסס eBPF, חלק מפרויקט Cilium. תיעוד וכלים זמינים ב-GitHub: https://github.com/cilium/hubble.